→ Write a function to return the product of two numbers.

→ Pascal triangle print. (use function). H.W

→ Write a function, if given a number return true if the number is an armstrong number, otherwise false.

→ write a program to print all the prime numbers from 1 to N.

→ Public / Private / Protected (✓) → study:
↳ access Specifier

Second Class of function

Variable naming
Pascal Convention

↳ variable name
↳ Second Class Of function

public class Main {

public int function (int a, int b) {
↳ datatype
↳ func^n name

→ int
→ Boolean
→ float
→ double
→ char
→ String

↳ Convention
↳ camelCase Convention

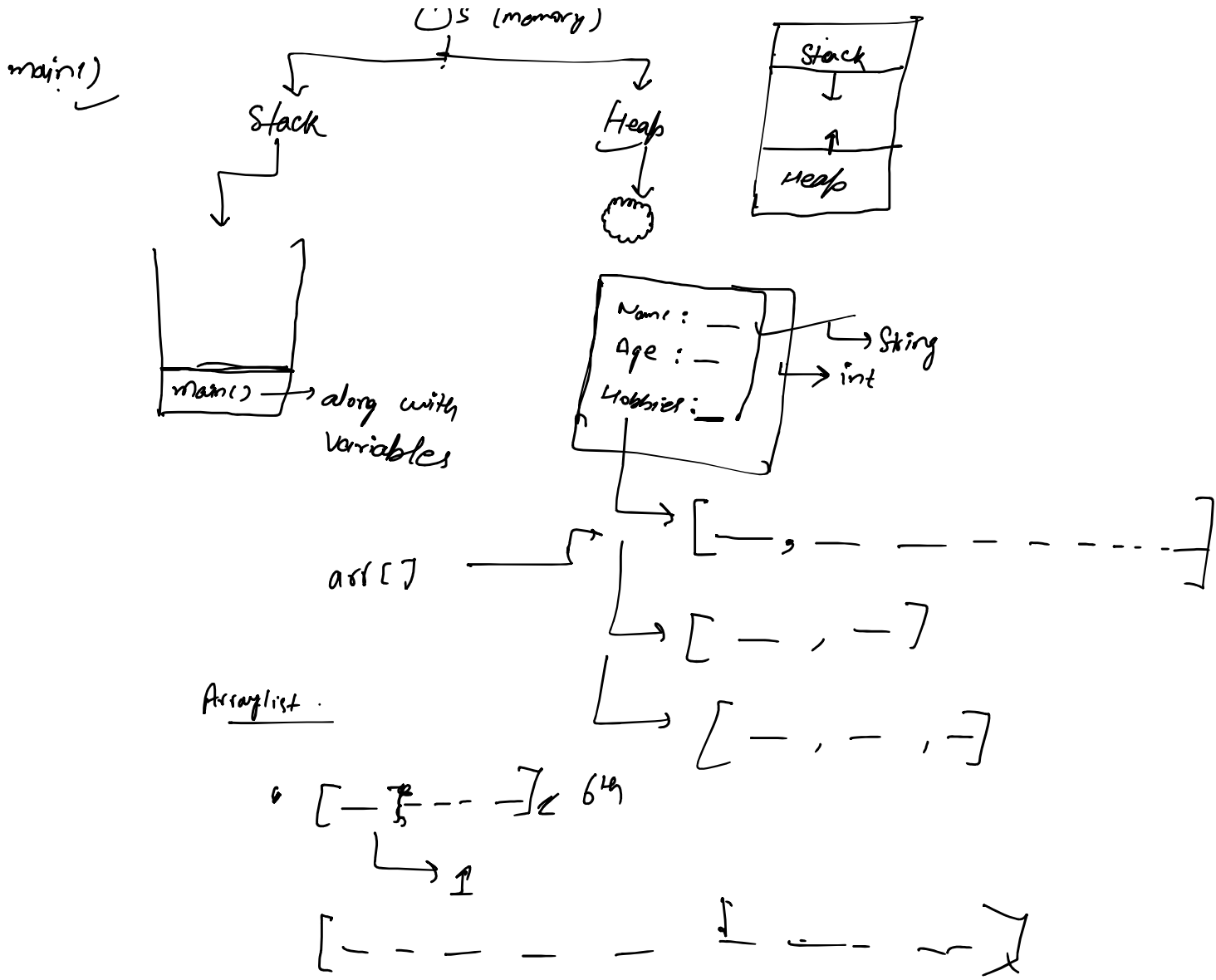
public / private
↳ access Specifier

↳ Second Class Of Functions
↓ small
↓ capital

main()

OS (memory)

Stack




```
public int function (int a, int b){
    access specifier
}
```

→ function to return the addition of two numbers.

```
public Class Main{
    public static int add (int a, int b){
```

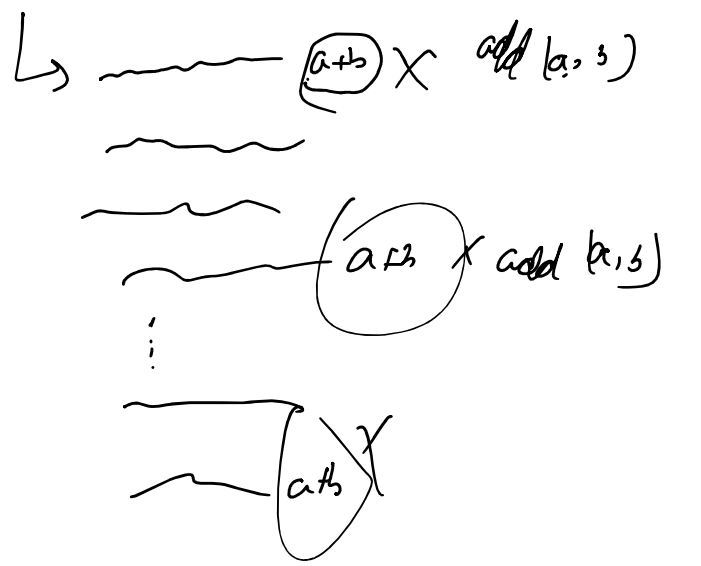
```
public static int add (int a, int b){
    return a+b; // Sout(a+b);
}

public static void main( ){
    int c = 5, b = 6;
    int sum = add (c, b);
    Sout (sum);      // O/P = 11
}
```

```
Main obj;

obj.main ;
```

//DRY




```
public static int product (long a, long b){
    return a*b
}
```

long

INT_MIN<a, b> INT_MAX

int b = 6;

long a = b;

b = a; (x)

Q Armstrong number

132 → 3

$1^3 + 3^3 + 2^3$

$= 1 + 27 + 8$

$= 36$

132 = 3

$1^3 + 3^3 + 2^3$

$= 36$

153

total = 3

$1^3 + 5^3 + 3^3$

$= 1 + 125 + 27$

$= 153$

func^n → true / false (Boolean)

$\frac{153}{10} = 15$

int num = 153;

int duplicate = num; count = 0;

while (duplicate != 0){

num	duplicate
153	15~~3~~ → count = 1
	15

```
while (duplicate != 0){
    count++;
    duplicate = duplicate / 10;
}
```

1 5 → count = 2

1 → count = 3

= 0

```
count = 0
duplicate = num;
int sum = 0;
while (duplicate > 0){
    int digit = duplicate % 10;
    sum = sum + Math.pow(digit, count);
    duplicate /= 10;
}
return num == sum;
```

```
public class Main {
  public static Boolean isArmstrong(int num){
    int duplicate=num;
    int count=0;
    while(duplicate>0){
      count++;
      duplicate/=10;
    }
    duplicate=num;
    int sum=0;
    while(duplicate>0){
      int digit=duplicate%10;
      sum=sum+(int)Math.pow(digit,count);
      duplicate/=10;
    }
    return num==sum;
  }
  public static void main(String[] args) {

    System.out.println(isArmstrong(127));
  }
}
```

return

num

returns double by default

num = 153

duplicate = 153

0

1

2

3

153

15

1

0

$Sum = 1^3 + 5^3 + 3^3$

153 syntax

0

$Sum = 0 + 3^3$

$Sum = 3^3$

Math.pow (base, power)

(base)^power

2nd 15

$Sum = 3^3 + 5^3$

3rd $Sum = 3^3 + 5^3 + 1^3$

num = 0

num = 153

Sum = 153

num = 153

Sum = 153

True / take

```
if(sum == num) return true;
else return false;
```